Project – High Level Design
on
Dockerized Construction Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	SHIWANI PATHAK	EN22CS301921
2.	SHRISHTI SAHU	EN22CS301936
3.	SHREYAS	EN22CS301934
4.	SANSKAR JAIN	EN22CS301868
5	RAJVEER SINGH CHANGAL	EN23CS3T1009

Group Name: Group 03D08

Project Number: DO-38

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025 - 2026

Table of Contents

1. Introduction
   - 1.1 Scope of the Document
   - 1.2 Intended Audience
   - 1.3 System Overview
2. System Design
   - 2.1 Application Design
   - 2.2 Process Flow
   - 2.3 Information Flow
   - 2.4 Components Design
   - 2.5 Key Design Considerations
   - 2.6 API Catalogue
   - 2.7 Dockerfile Design
   - 2.8 Docker Compose Design
   - 2.9 Deployment Workflow
3. Data Design
   - 3.1 Data Model
   - 3.2 Data Access Mechanism
   - 3.3 Data Retention Policies
   - 3.4 Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1 Security Aspects
   - 7.2 Performance Aspects
8. DevOps & Deployment Architecture
9. References

1. Introduction

1.1 Scope of the Document

This High-Level Design (HLD) document describes the overall architecture, system components, workflows, and design decisions of the Dockerized Construction Planning Service.

The document covers:

- Application architecture
- Containerization strategy
- Kubernetes orchestration
- AWS deployment structure
- CI/CD pipeline design
- System scalability and performance considerations

It provides a conceptual and architectural overview without going into implementation-level details (covered in LLD).

1.2 Intended Audience

This document is intended for:

- DevOps Engineers
- Cloud Engineers
- Software Developers
- Academic Evaluators
- System Architects

1.3 System Overview

The system is a **containerized web-based construction planning platform** that generates optimized construction schedules using backend AI logic.

Key Characteristics:

- Microservice-based architecture
- Fully containerized using Docker
- Orchestrated using Kubernetes (K3s)

- Deployed on AWS EC2
- CI/CD enabled using GitHub Actions

Core Components:

- **Frontend**: Static UI served via Nginx
- **Backend**: Flask API with planning logic
- **Containerization Layer**: Docker
- **Orchestration Layer**: Kubernetes
- **Infrastructure**: AWS EC2
- **Automation**: CI/CD pipeline
-

2. System Design

2.1 Application Design

The system follows a **layered microservices architecture**:

User → Browser → Frontend (Nginx)
↓
Backend (Flask API)
↓
AI Processing Layer

Design Principles:

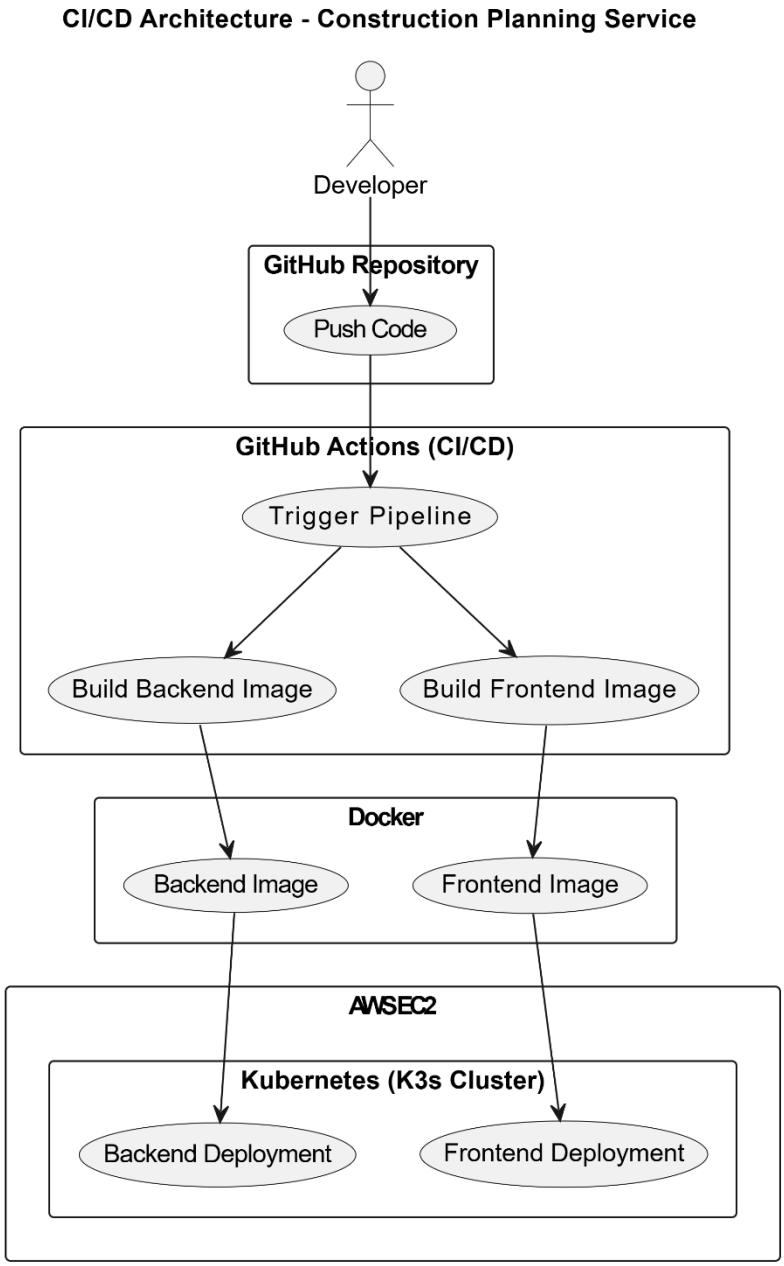
- Separation of concerns
- Stateless services
- Loose coupling between components
- Scalable container-based architecture

2.2 Process Flow

1. User accesses the application via browser
2. Request reaches frontend (NodePort exposed service)
3. Frontend sends HTTP request to backend API
4. Backend processes request using:
    - Planner Agent

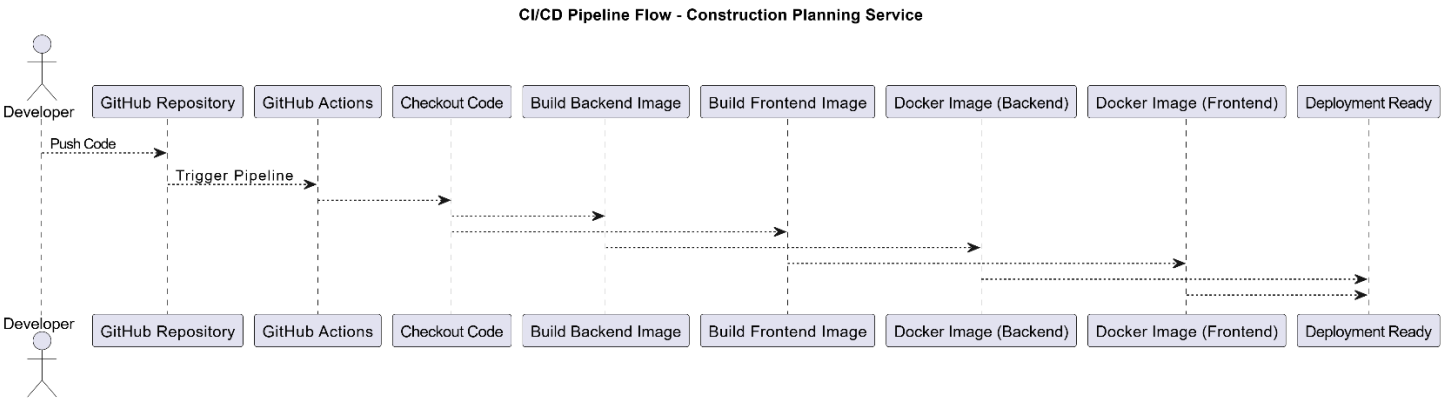
- Scheduler
- Risk Analyzer

5. Backend generates:
    - Optimized schedule
    - Risk analysis
    - Gantt chart
6. Response returned to frontend
7. UI renders results



2.3 Information Flow

User → Browser → Frontend Container → Backend Container → Response → UI



2.4 Components Design

Frontend Service

- Built using HTML, CSS, JavaScript
- Served via Nginx container
- Exposed using Kubernetes NodePort
- Communicates with backend via HTTP

Backend Service

- Developed using Flask (Python)
- Provides REST API endpoints
- Handles:
    - Task planning
    - Scheduling
    - Risk analysis
- Generates dynamic Gantt chart

Docker (Containerization Layer)

- Packages application into images
- Ensures environment consistency
- Enables portability across systems

Docker Images

- Backend Image:
  - Base: python:3.11-slim
  - Lightweight and optimized
- Frontend Image:
  - Base: nginx:alpine
  - Fast static serving

Kubernetes (K3s)

- Manages container orchestration
- Handles:
  - Pod lifecycle
  - Service discovery
  - Load balancing
  - Self-healing

Kubernetes Components Used

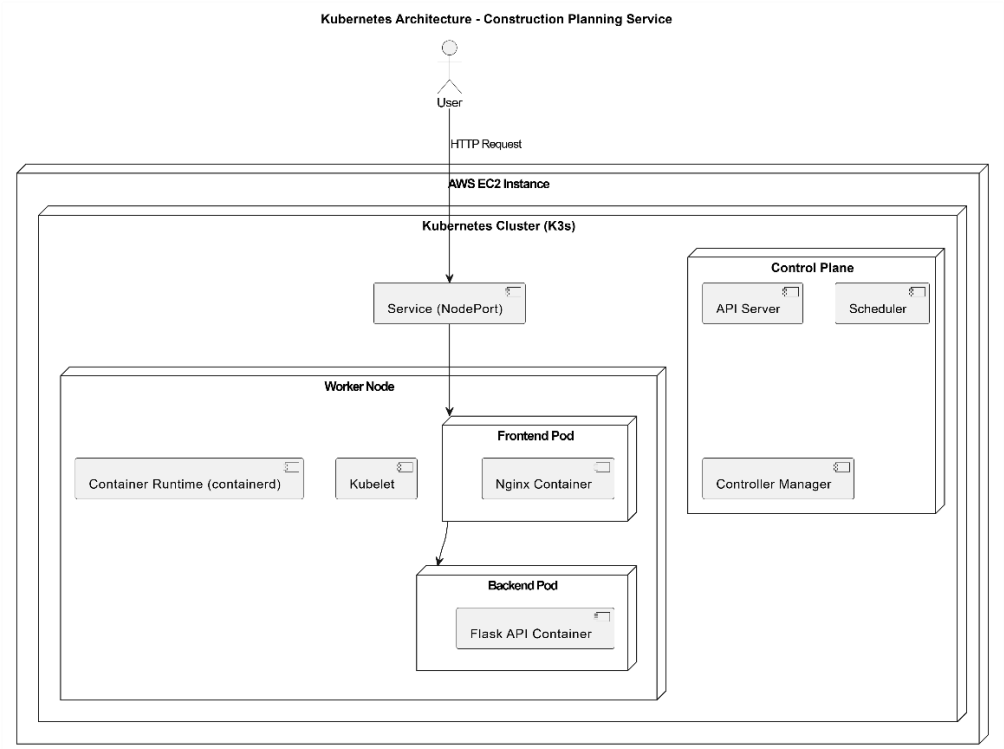
- **Pods** → Running containers
- **Deployments** → Manage replicas
- **Services (NodePort)** → External access
- **Cluster** → Single-node K3s cluster on EC2

AWS EC2 Infrastructure

- Ubuntu-based virtual machine
- Hosts Kubernetes cluster
- Provides public IP for access

CI/CD (GitHub Actions)

- Automates build process
- Triggered on code push
- Builds Docker images automatically



2.5 Key Design Considerations

Containerization

- Eliminates environment mismatch
- Standardized deployment

Scalability

- Kubernetes allows horizontal scaling
- Independent scaling of frontend/backend

High Availability

- Kubernetes self-healing ensures uptime

Security

- Use of .env for sensitive data
- Controlled port exposure
- Network isolation

Portability

- Runs on:
    - Local machine

- AWS EC2
- Kubernetes cluster

Performance Optimization

- Lightweight Docker images
- Efficient API communication

2.6 API Catalogue

POST /plan

Description: Generates optimized construction plan

Request:

```
{
  "goal": "Build house"
}
```

Response:

```
{
  "goal": "Build house",
  "optimized_schedule": [...],
  "risk": "...",
  "severity": "Medium"
}
```

GET /gantt.png

Returns generated Gantt chart image

2.7 Dockerfile Design

Backend Dockerfile

Key Instructions:

- FROM python:3.11-slim → lightweight base image
- WORKDIR /app → working directory
- COPY requirements.txt → dependency optimization
- RUN pip install → installs dependencies

- COPY backend → copies application
- EXPOSE 5000 → exposes API port
- CMD → runs Flask server

Frontend Dockerfile

- Uses nginx:alpine
- Serves static files
- Exposes port 80

2.8 Docker Compose Design

Purpose

- Orchestrate multiple containers
- Define services and networking

Services

Service	Role
backend	API server
frontend	UI

Features

- Automatic networking
- Service dependency management
- Single command deployment

2.9 Deployment Workflow

Local Deployment

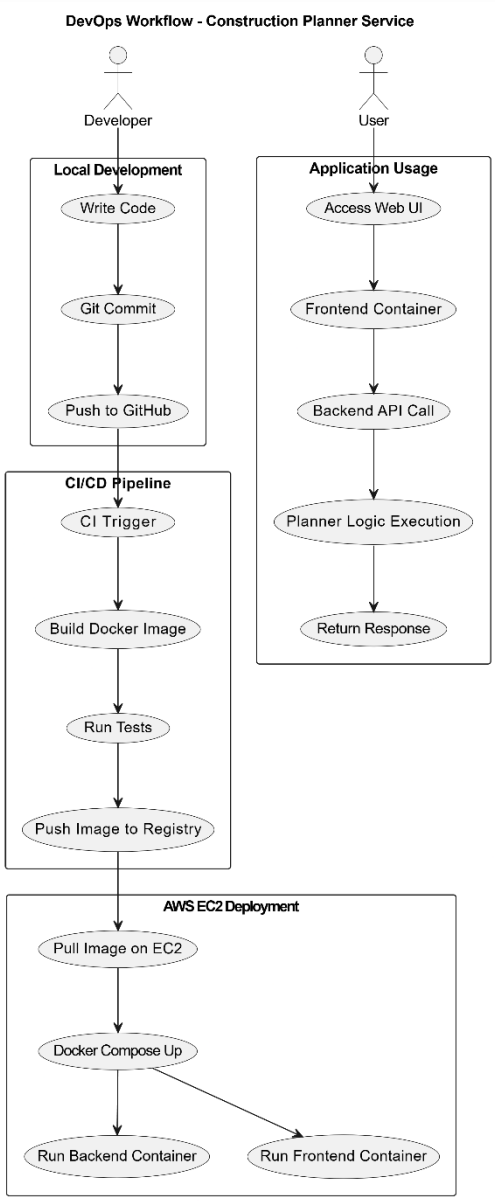
docker compose up --build

AWS Deployment

1. Launch EC2 instance
2. Install Docker
3. Clone repository
4. Run:

docker-compose up --build -d

Access Application : http://EC2_IP:3000



3. Data Design

3.1 Data Model

- Input: Goal (string)
- Output:
  - Task list
  - Dependencies
  - Schedule
  - Risk assessment

- Gantt chart

3.2 Data Access Mechanism

- REST API communication (JSON format)
- Stateless interaction

3.3 Data Retention Policies

- No persistent storage
- Temporary data stored in memory

3.4 Data Migration

- Not applicable (stateless system)

4. Interfaces

External Interfaces:

- Browser UI
- REST API

Internal Interfaces:

- Frontend → Backend communication
- Kubernetes service networking

5. State and Session Management

- Stateless architecture
- No session persistence
- Each request handled independently

6. Caching

- Minimal caching used
- Gantt chart regenerated per request
- Browser caching for static assets

7. Non-Functional Requirements

7.1 Security Aspects

- CORS enabled for safe communication
- Controlled network access via Kubernetes
- Environment variable protection

7.2 Performance Aspects

- Fast response due to lightweight containers
- Kubernetes ensures resource optimization
- Efficient API handling

8. DevOps & Deployment Architecture

Docker Workflow

- Build images using Dockerfiles
- Run containers locally or via Compose

Kubernetes Workflow

- Deploy using YAML files
- Manage services and pods
- Enable scalability and self-healing

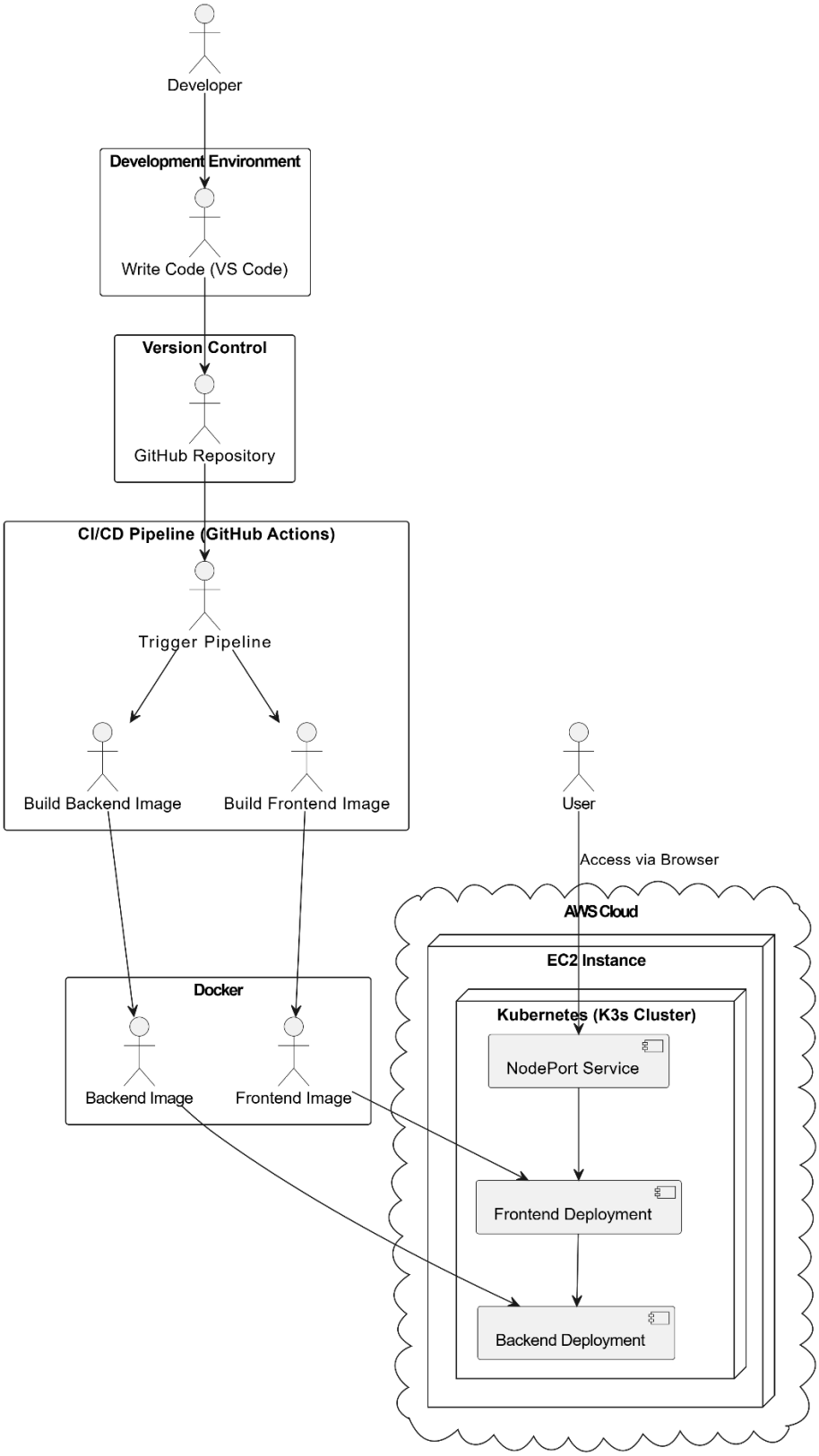
AWS Deployment

- EC2 instance hosts Kubernetes cluster
- Public IP used for access

CI/CD Workflow

- Code pushed to GitHub
- GitHub Actions triggers pipeline
- Docker images built automatically

DevOps & Deployment Architecture - Construction Planning Service



9. References

[1] Docker Inc., *Docker Documentation*. [Online]. Available: https://docs.docker.com/
[Accessed: Mar. 23, 2026].

[2] Docker Inc., *Docker Compose Documentation*. [Online]. Available:
https://docs.docker.com/compose/
[Accessed: Mar. 23, 2026].

[3] The Linux Foundation, *Kubernetes Documentation*. [Online]. Available:
https://kubernetes.io/docs/
[Accessed: Mar. 23, 2026].

[4] Rancher Labs, *K3s Lightweight Kubernetes Documentation*. [Online]. Available:
https://docs.k3s.io/
[Accessed: Mar. 23, 2026].

[5] Amazon Web Services, *Amazon EC2 Documentation*. [Online]. Available:
https://docs.aws.amazon.com/ec2/
[Accessed: Mar. 23, 2026].

[6] Amazon Web Services, *What is DevOps?*. [Online]. Available:
https://aws.amazon.com/devops/what-is-devops/
[Accessed: Mar. 23, 2026].

[7] Git SCM, *Git Documentation*. [Online]. Available: https://git-scm.com/docs
[Accessed: Mar. 23, 2026].

[8] GitHub Inc., *GitHub Actions Documentation*. [Online]. Available:
https://docs.github.com/en/actions
[Accessed: Mar. 23, 2026].

[9] Pallets Projects, *Flask Documentation*. [Online]. Available:
https://flask.palletsprojects.com/
[Accessed: Mar. 23, 2026].

[10] Python Software Foundation, *Python Documentation*. [Online]. Available:
https://docs.python.org/3/
[Accessed: Mar. 23, 2026].

[11] NGINX Inc., *NGINX Documentation*. [Online]. Available: https://nginx.org/en/docs/
[Accessed: Mar. 23, 2026].